Computing for Analytical Work

Session 3 — What broke?

Block 5 — Reading errors and debugging systematically

Knowledge check 3 — 10 minutes, closed book, 3 questions

- Covers **Session 2 and Homework 2**: interpreters, environments, `uv`, notebook state, Git checkpoints
- Short answers, in your own words
- Diagnostic, not tricky — if you did the work, this is easy
- Closed book means closed: no laptop, no phone, no notes
- **This block's lecture is 35 minutes**, not 45 — the check comes out of lecture, never out of lab
- Hand it in when the timer ends; the lecture starts immediately after

This block in one sentence

Error messages are evidence. Debugging is a method, not a vibe.

A traceback is not a punishment. It is the machine telling you, precisely, what it could not do and where it gave up.

Agenda

1. What a traceback is, and the two anchors
2. Symptom versus cause
3. Six errors you will meet this year
4. Inspect before you guess: print, minimal example, asking well
5. Git thread — `git log`: what happened before?
6. AI thread — a prompt made of evidence
7. Lab 5 — The traceback is the map

What a traceback is

- Python hit something it could not do, and stopped
- On its way out, it wrote down the **chain of calls** that led there
- It is printed **top to bottom**: the outermost call first, the failure last
- That is why the useful part is at the **bottom**
- Nothing about it is random; every line points at a file and a line number

First: find it in the scroll

- The traceback begins at the literal line `Traceback (most recent call last):` — that is what you search for
- Scroll back: trackpad, scrollbar, or **Cmd-F / Ctrl-F in VS Code's terminal**
- Several runs' output stacked up? You cannot tell which error is *this* run's
- The habit that removes the problem: **`clear` before every rerun** — then everything on screen is from this run

The two anchors

1. **Last line** → the **error type** → *what kind of problem*
2. **Nearest line above that is in YOUR file** → the **failing line** → *where*

Everything else is the call stack. Scroll up later, if the two anchors are not enough.

A real one

```
Traceback (most recent call last):
  File "/Users/anna/lab05/scripts/report.py", line 14, in <module>
    total = monthly_total(df)
  File "/Users/anna/lab05/scripts/report.py", line 8, in monthly_total
    return df["revenue"].sum()
  File ".../site-packages/pandas/core/frame.py", line 4102, in __getitem__
    indexer = self.columns.get_loc(key)
  File ".../site-packages/pandas/core/indexes/base.py", line 3812, in get_loc
    raise KeyError(key) from err
KeyError: 'revenue'
```

The same traceback, annotated

```
  File ".../scripts/report.py", line 8, in monthly_total   <- ANCHOR 2: where
    return df["revenue"].sum()                              <- the failing line
  File ".../site-packages/pandas/core/frame.py" ...        <- call stack (pandas)
  File ".../site-packages/pandas/core/indexes/base.py" ... <- call stack (pandas)
KeyError: 'revenue'                                          <- ANCHOR 1: what
```

- **What:** `KeyError` — a name was looked up and did not exist
- **Where:** `report.py` line 8, `df["revenue"]`
- **Next:** `print(df.columns)` — what names *does* the dataframe have?

When the library is in the way

- Frames in `site-packages/` belong to pandas, numpy, matplotlib — **not to you**
- The bug is almost never inside the library
- Walk **up** from the bottom until the path is a file in **your** project
- That frame is anchor 2, even if it is four lines above the error
- Notebook tracebacks look different but obey the same rule: find the cell, find the line

Symptom versus cause

- **Symptom:** what the traceback says — `KeyError: 'revenue'`
- **Cause:** why the world is that way — the CSV header says `Revenue`, capital R
- The traceback gives you the symptom for free; the cause you have to go find
- Fixing the symptom: `df["Revenue"]` — works until the next file
- Fixing the cause: normalise column names right after loading
- The diagnosis note asks for both, on purpose

`FileNotFoundError` and `ModuleNotFoundError`

`FileNotFoundError: [Errno 2] No such file or directory: 'data/sales.csv'`

- Usually means: the path is relative to the **wrong working directory**, or the file is in `data/raw/`
- Inspect first: `pwd`, `ls data/`, `Path.cwd()` in the notebook

`ModuleNotFoundError: No module named 'pandas'`

- Usually means: you are running a **different Python** than the one you installed into
- Inspect first: `uv run python -c "import sys; print(sys.executable)"`

`NameError` and `KeyError`

`NameError: name 'df' is not defined`

- Usually means: the line that creates `df` never ran — typo, skipped cell, wrong order
- Inspect first: search the file for `df =`; in a notebook, Restart and Run All

`KeyError: 'revenue'`

- Usually means: a column, dictionary key, or config entry is **spelled differently** than you think
- Inspect first: `print(df.columns.tolist())` or `print(d.keys())`

`ValueError` and `TypeError`

```
ValueError: could not convert string to float: '€1,200'
```

- Usually means: the **type is right, the content is wrong** — a number with a currency sign, a date with the wrong format
- Inspect first: look at the actual values — `df["amount"].head()`, `df["amount"].unique()[:10]`

```
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

- Usually means: you passed the **wrong kind of thing** — a string where a number was expected, a list where a dataframe was expected
- Inspect first: `type(x)`, `df.dtypes`

Print, log, inspect — before guessing

- Guessing is editing code you have not looked at, to fix data you have not looked at
- One `print()` right above the failing line beats ten minutes of theorising
- Print the thing the line uses: `print(df.columns)`, `print(type(x))`, `print(path.exists())`
- Then run it again, with the same command that failed
- Remove the print when done; a diagnosis note is where the finding lives

Minimal reproducible example

```python
import pandas as pd
df = pd.read_csv("data/raw/sales.csv")
print(df.columns.tolist())
print(df["revenue"].sum())      # the line that fails, isolated
```

- Strip everything that is not needed to produce the error
- If four lines still fail, the bug is in those four lines
- If four lines work, the bug is in what you removed — bisect
- This is also the best thing to paste when asking for help

How to ask for help well

A question a TA, a colleague, or an assistant can actually answer contains:

1. The **exact** error, pasted, not paraphrased
2. The **failing line** and the file it lives in
3. Where things are: file tree, working directory, the command you ran
4. What you **expected** and what **happened** instead
5. What you **already tried** — and what it showed you

"It doesn't work" contains none of these.

Git thread — what happened before?

`git log` — the history of your checkpoints

```
git log --oneline
```

```
e7c21a4 Rename revenue column after load
b93f0d2 Add monthly_total()
a41f9c2 Initial working report
```

- Every commit is a moment the project was in a known state
- Small commits mean small diffs between working and broken

If it worked before, history tells you what changed

```
git log --oneline -5 -- scripts/report.py
git diff a41f9c2 -- scripts/report.py
```

- Which commits touched this file? That is the suspect list
- What is different between the last working commit and now? That is the diff
- You are not guessing what you changed; you are **reading** it
- Lab 5 ships with a small history — use it

AI thread — a prompt made of evidence

"Fix this" versus evidence

Bad prompt

```
my script doesn't work fix this
```

Good prompt

```
Running `uv run python scripts/report.py` from the project root gives
KeyError: 'revenue' at scripts/report.py line 8: df["revenue"].sum().
print(df.columns.tolist()) shows ['date', 'Revenue', 'region'].
I expected a column named 'revenue'. I have not changed the CSV.
What are the most likely causes, and what should I inspect next?
```

AI gets dramatically better when you provide evidence instead of frustration

- The five ingredients from "asking well" are the prompt
- Ask for **likely causes and what to inspect**, not "fix it"
- Try **one** change; run the **same** command; read the result yourself
- Then explain the fix in your own words — that sentence goes in the diagnosis note
- Your AI-use note names what you asked and how you verified the answer

5-minute stand-and-stretch

Stand up. Leave the laptop. Back in five for Lab 5.

Lab 5 — The traceback is the map

First 15 minutes: no AI. The traceback and the failing file only.

- Chat windows, AI editors, assistant tabs: **closed**. Search engines are fine
- Why: the reflex we are installing is *"check the columns"*, not *"ask what to check"*
- Only one of those reflexes survives a closed-book exam
- Solve it early? Write the four notes; the pair window is at minute 33. The window is a floor on struggle, not a ceiling on speed
- After 15 minutes, normal AI policy resumes

The lab: four failures in sequence

`scripts/report.py` fails **four times**. Each fix reveals the next.

1. A wrong **filename**
2. A wrong **column name**
3. A value that **will not convert**
4. A function called with the **wrong kind of input**

You already know which error type each one raises. Prove it.

Run it, diagnose it, note it

```
uv sync
uv run python scripts/report.py
```

- Run it **exactly like that**, from the project root, every time
- For each failure: error type, failing line, what you inspected, what you changed, how you verified
- **One diagnosis note per failure** — four five-part notes in `DIAGNOSIS.md`; part 3 quotes the two anchors
- Commit after each fix; `git log` should show four checkpoints
- **Minute 33:** pick one of the four and explain it to your neighbour, about a minute, no notes; the what-if is on the next slide

The last ten minutes — explain it to a neighbour

Speaker, about a minute, pointing at the screen, not reading the note: symptom → cause → evidence → change → verification.

Listener asks:

1. Show me the evidence.
2. Why did it fail — not just where?
3. Undo your first fix, in your head only, and run again. Which of the four errors do you see, and do the other three still exist?

Swap. Unsure, or you disagree? **Hand up.** Then the answer, for everyone. Before you leave: `DIAGNOSIS.md` to the Lab 5 checkpoint on Moodle.